ASSIGNMENT 5 – CO5

PART B | TASK B1

IMPLEMENT AND INSTRUMENT THE FILE STORE

Java Text-File Persistence, Validation and Performance Measurement

Course: Programming in Java
Topic: Data Persistence Engineering – Files and Databases

Implementation report with executed validation tests, line-count buffering benchmarks, search-growth measurements and interpretation. Student-identifying details are intentionally omitted.

1. Objective and Task Scope

B1 requires implementation of the Part A1 text-file bus-pass store and the buffered line counter from A2. It assesses validation, comparison with the earlier design, buffering performance on 1 MB, 10 MB and 50 MB files, and search-time growth from 1,000 to 100,000 records. The supplied B1 starter program explicitly stores five fields (pass ID, register number, name, route, amount), and its expected output returns five fields. The implementation follows that supplied executable specification.

Scope discrepancy to disclose

Part A1 describes seven fields (including boarding stop and validity date), whereas the supplied B1 PassStore starter code and expected output use five fields and omit boarding stop and validity date. To preserve the given starter program and its expected output, this B1 implementation uses the five fields in the supplied code. If the faculty expects the complete seven-field A1 schema in the final store, extend add/find/list and all generated records consistently with the two omitted fields.

2. B1.1 – Validate the Implementation

The TODO 1 in PassStore.find() is completed using BufferedReader within try-with-resources. Each line is split with split(",", -1), so empty trailing fields are retained. A line is accepted only when it has exactly five fields. Damaged rows are skipped and the scan continues; a matching pass ID returns its five fields, otherwise null is returned. listByRoute() similarly skips wrong-width rows and rows with a non-numeric route.

Test	Input / action	Observed result	What it verifies
Normal add + find	Add P1003 then find P1003	[P1003, 192311003, Divya, 7, 3800.0]	Append succeeds and lookup returns all five fields.
Missing ID	Find P9999	null	A missing key is reported without crashing.
Damaged line	Append P1002,DAMAGED LINE (2 fields)	find P1002 returns null; later valid P1003 still found	Wrong-width rows are skipped and do not stop the scan.
List by route	List route 7	One record: P1003	Route filtering returns matching valid rows.

The supplied main() expected output was reproduced exactly:

```
[P1003, 192311003, Divya, 7, 3800.0]
null
```

3. B1.2 – Compare with Part A Design

The implementation follows the A1 text-store idea of appending records and scanning sequentially for find/list operations. It uses BufferedWriter for append and BufferedReader for reading, both inside try-with-resources. A practical validation rule was made explicit: exactly five comma-separated fields are required, and malformed rows are skipped rather than terminating the scan. The starter's simple comma-separated format does not escape commas inside names; therefore it is safe only for values that do not contain commas. A robust extension would use quoted CSV parsing or a delimiter and escaping rule agreed in the file format design.

4. B1.3 – Measure the Effect of Buffering

The same files were read using FileInputStream.read() one byte at a time and BufferedInputStream with an 8,192-byte buffer. Each version was warmed up once and timed three times; the median elapsed time is reported. File sizes are binary MiB (1 MiB = 1,048,576 bytes), matching the generated exact byte sizes.

File size	Bytes	Unbuffered median	Buffered median	Speed-up
1 MiB	1,048,576	215.032 ms	7.001 ms	30.7×
10 MiB	10,485,760	2,015.105 ms	65.127 ms	30.9×
50 MiB	52,428,800	10,547.707 ms	306.928 ms	34.4×

Interpretation

Buffering reduced measured time by roughly 31–34× in this run. The unbuffered version incurs per-byte Java stream read overhead; the buffered version refills its 8 KiB buffer in larger chunks. For a 50 MiB file, the approximate number of buffer refills is 52,428,800 / 8,192 = 6,400, instead of 52,428,800 calls to read() at the Java API level. A Java read() call is not necessarily one physical disk access or one kernel syscall: the operating system page cache and JVM implementation affect the underlying I/O

behaviour. Both algorithms are O(N) in bytes; buffering reduces the constant overhead and uses O(B) extra memory, where B is the buffer size.

5. B1.4 – Measure Growth in Find Time

A pass file was generated at each size. The target pass was deliberately placed at the end of the file, representing a worst-case successful sequential scan. Five searches were timed and the median was recorded.

Records	Target location	Median find time
1,000	Last record	2.169 ms
10,000	Last record	9.853 ms
50,000	Last record	12.167 ms
100,000	Last record	21.397 ms

Interpretation

Find time generally increases as the file grows because each lookup scans from the beginning until the target is reached. The expected complexity is O(n) per search, with O(1) extra application memory apart from the returned row and I/O buffers. The increase is not perfectly linear because results are affected by filesystem cache, JVM warm-up, scheduling and measurement noise. A HashMap built once at startup would take O(n) time and O(n) memory to build, then average O(1) per lookup; for q searches the total is O(n + q), compared with O(q × n) for repeated scans.

6. Environment and Reproducibility

Execution environment used for the recorded measurements: OpenJDK 21.0.12.1 (64-bit Server VM), Linux x86_64, kernel 6.18.57. The values are measurements from this execution environment, not universal performance guarantees. Re-run the benchmark on the student's own computer and replace or supplement these numbers with that machine's results before final submission if the instructor expects student-local measurements.

7. Conclusion

The supplied TODO 1 is completed and the expected output is reproduced. The tests show that valid records can be added, existing records can be found, absent IDs return null, route listing filters records, and a wrong-width damaged line is skipped without stopping later searches. Buffering substantially reduces measured line-count time while preserving O(N) time complexity. Sequential find remains O(n) and becomes slower as the file grows; a HashMap index trades O(n) startup time and O(n) memory for average O(1) lookup.

8. Submission Evidence Checklist

Include the Java sources, generated test input, captured console outputs, benchmark output text, and screenshots from your own VS Code/terminal execution. The assignment states that screenshots alone are insufficient; retain the measurement tables and explanations in this report. Add the working GitHub repository link to the report cover page before submission.